

Print all the Nodes at a distance of K in a Binary Tree

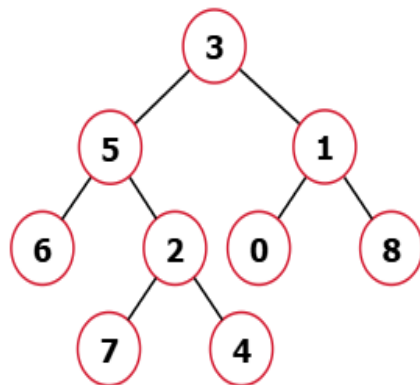
Problem Statement: Given the root of a binary tree, the value of a target node `target`, and an integer `k`. Return an array of the values of all nodes that have a distance `k` from the target node. The answer can be returned in any order (N represents null).

Examples

Input: `root = [3, 5, 1, 6, 2, 0, 8, N, N, 7, 4]` , `target = 5`, `k = 2`

Output: `[7, 4, 1]`

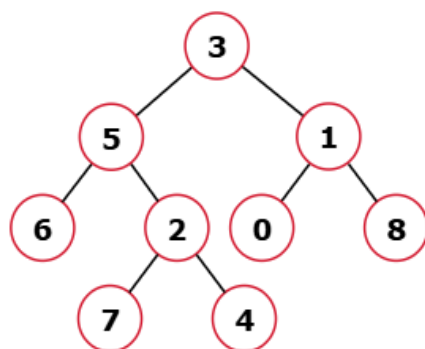
Explanation: The nodes that are a distance 3 from the target node (with value 5) have values 7 , 4 and 1.



Input :`root = [3, 5, 1, 6, 2, 0, 8, N, N, 7, 4]` , `target = 5`, `k = 3`

Output :`[0, 8]`

Explanation : The nodes that are a distance 3 from the target node (with value 5) have values 0, 8.



Approach

Algorithm

In a binary tree, each node only has references to its children. To find all nodes at a specific distance `K` from a given `target` node, we must be able to move both **downward** (to children)

and **upward** (to parent). Since the binary tree doesn't store parent links, we simulate this by converting the tree into an **undirected graph**. This allows traversal in all directions: left, right, and parent. Once we treat the tree as a graph, we perform a standard **Breadth-First Search (BFS)** starting from the target node and collect all nodes that are exactly K steps away.

1. Build the Parent Map:

Traverse the entire binary tree using either BFS or DFS and map each node to its parent node. This step effectively adds upward connectivity to the tree, allowing access to parent nodes during traversal.

- Create a **parentMap** where each key is a child node and its value is the corresponding parent node.
- This transforms the binary tree into an undirected structure where each node can move left, right, or up.

2. Perform BFS from the Target Node:

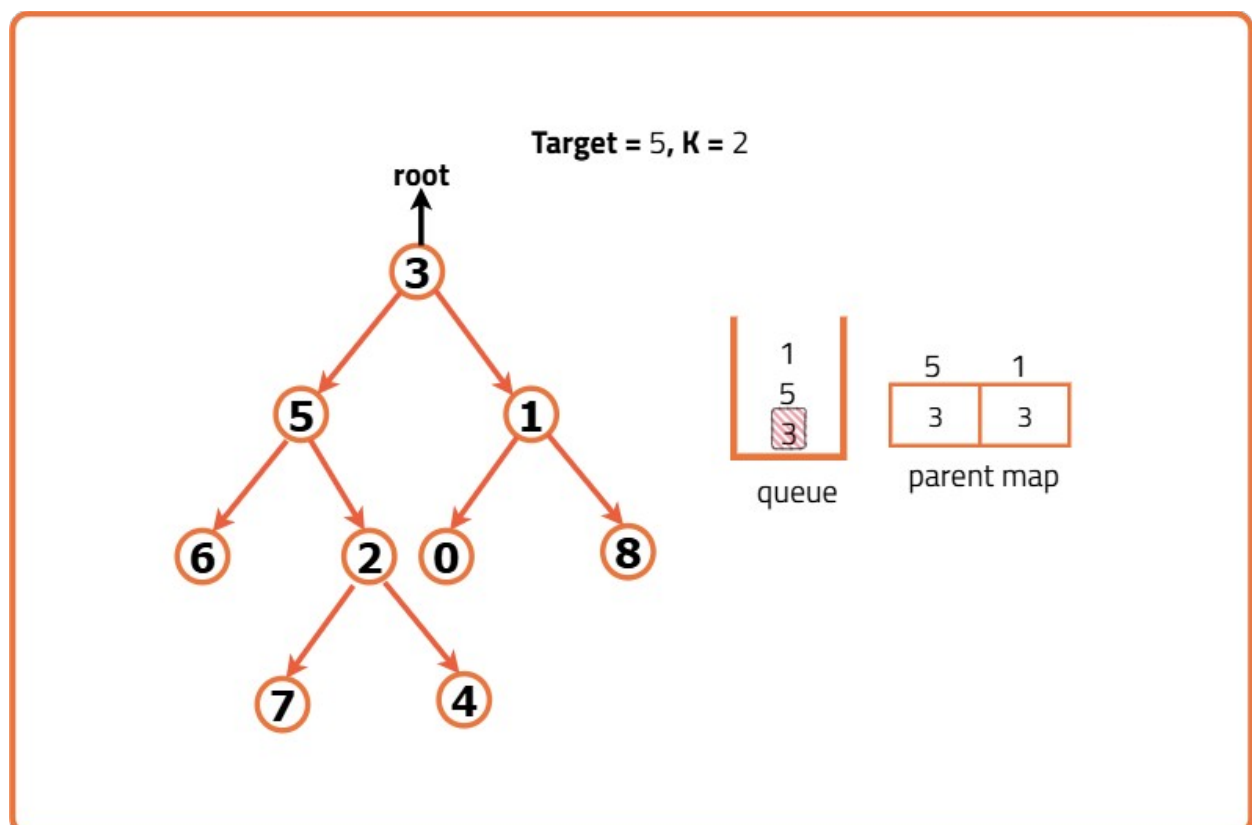
Begin a level-order (BFS) traversal from the target node to explore all nodes at increasing distances.

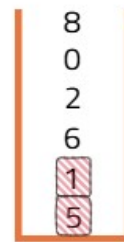
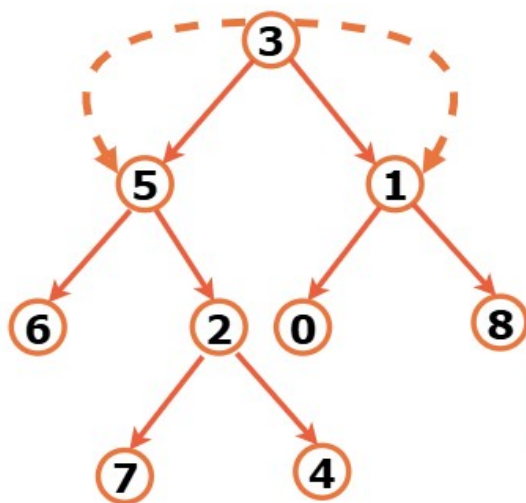
- Initialize a queue with the target node and a set to track visited nodes.
- For each node at the current BFS level:
 - Visit the left child if it exists and hasn't been visited.
 - Visit the right child if it exists and hasn't been visited.
 - Visit the parent (from the **parentMap**) if it exists and hasn't been visited.
- Repeat the process level by level until distance K is reached.

3. Store the Result:

When the BFS reaches distance K, all nodes remaining in the queue are exactly K edges away from the target node.

- Store the values of these nodes in a result vector and return it.

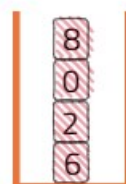
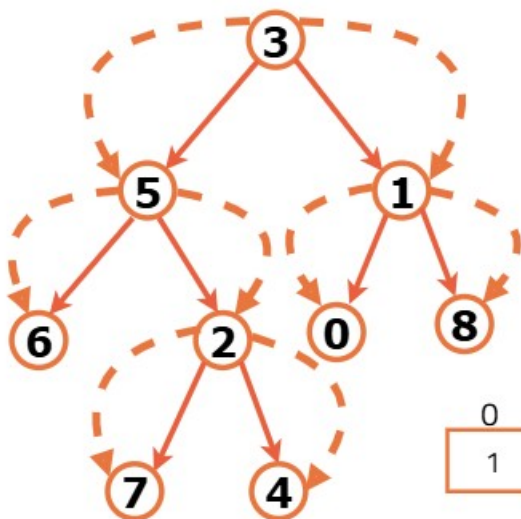




queue

0	1	2	5	6	8
1	3	5	3	5	1

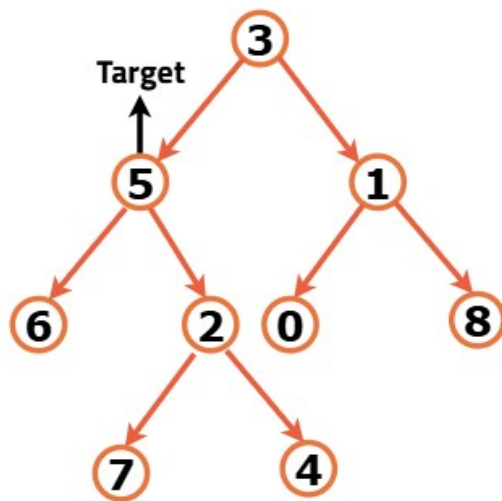
parent map



queue

0	1	2	4	5	6	7	8
1	3	5	2	3	5	2	1

parent map

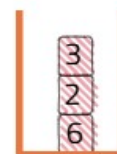
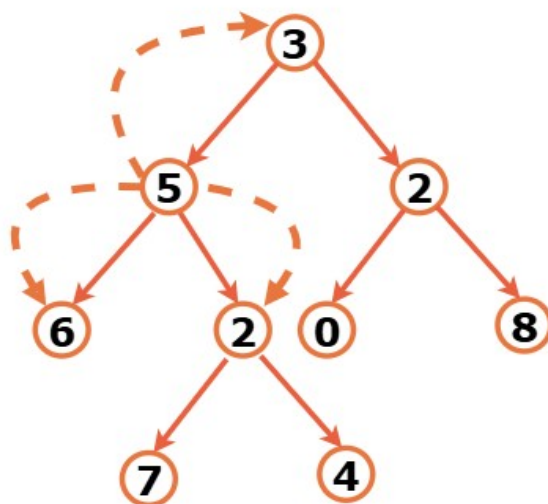


queue



visited

current level = 0

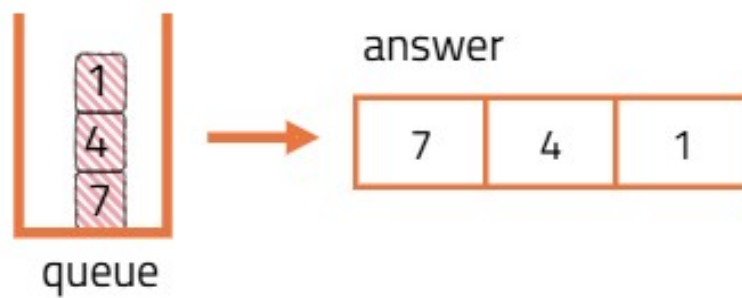
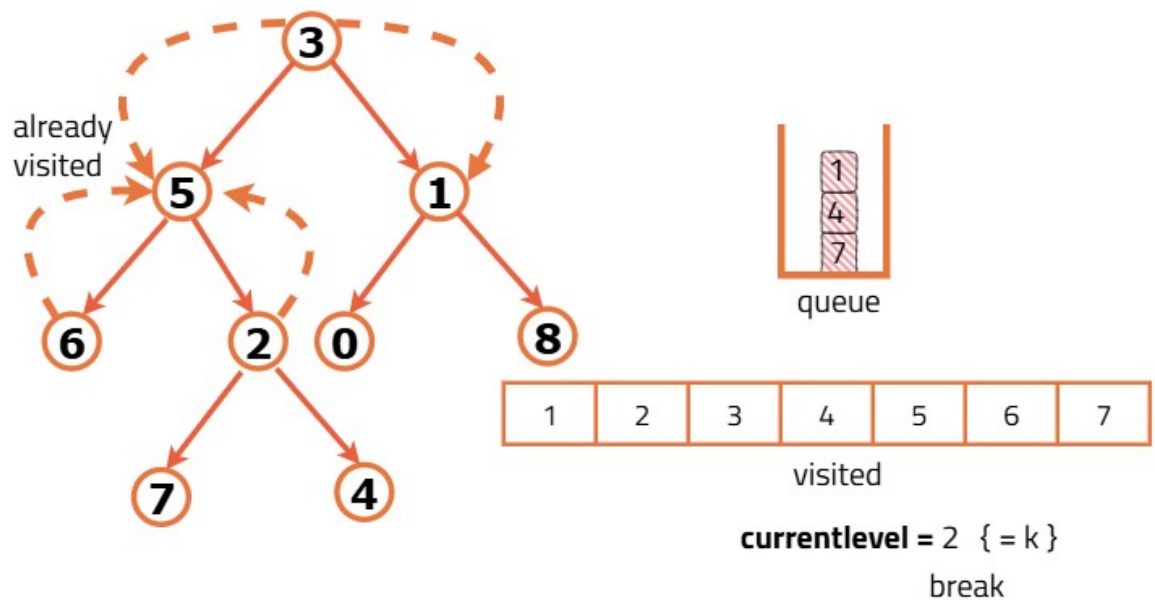


queue



visited

current level = 1



Code

```
import java.util.*;

// Definition for a binary tree node
class TreeNode {
    int val;
    TreeNode left, right;

    TreeNode(int x) {
        val = x;
        left = right = null;
    }
}

class Solution {

    // Main method to find nodes at distance K from target node
    public List<Integer> distanceK(TreeNode root, TreeNode target, int k) {
        if (root == null) return new ArrayList<>();

        // Step 1: Build parent references using BFS
        Map<TreeNode, TreeNode> parentMap = new HashMap<>();
        mapParents(root, parentMap);

        // Step 2: Perform BFS from target to find all nodes at distance K
        return bfsFromTarget(target, parentMap, k);
    }

    // Helper method to build the parent map using level-order traversal
    private void mapParents(TreeNode root, Map<TreeNode, TreeNode> parentMap) {
        Queue<TreeNode> queue = new LinkedList<>();
        queue.offer(root);

        while (!queue.isEmpty()) {
            TreeNode node = queue.poll();

            // Record left child's parent
            if (node.left != null) {
                parentMap.put(node.left, node);
                queue.offer(node.left);
            }

            // Record right child's parent
            if (node.right != null) {
                parentMap.put(node.right, node);
                queue.offer(node.right);
            }
        }
    }

    // BFS starting from the target node to collect nodes at distance K
    private List<Integer> bfsFromTarget(TreeNode target, Map<TreeNode, TreeNode>
parentMap, int k) {
        Queue<TreeNode> queue = new LinkedList<>();
        Set<TreeNode> visited = new HashSet<>();
        queue.offer(target);
        visited.add(target);

        int currentLevel = 0;

        // Standard level-order BFS
        while (!queue.isEmpty()) {
            int size = queue.size();
            List<Integer> result = new ArrayList<>();
            for (int i = 0; i < size; i++) {
                TreeNode node = queue.poll();
                if (node.val == target.val) result.add(node.val);
                if (node.left != null) {
                    if (!visited.contains(node.left)) {
                        queue.offer(node.left);
                        visited.add(node.left);
                    }
                }
                if (node.right != null) {
                    if (!visited.contains(node.right)) {
                        queue.offer(node.right);
                        visited.add(node.right);
                    }
                }
            }
            if (currentLevel == k) return result;
            currentLevel++;
        }
        return result;
    }
}
```

```

        // Stop traversal once we reach distance K
        if (currentLevel++ == k) break;

        for (int i = 0; i < size; i++) {
            TreeNode node = queue.poll();

            // Explore left child
            if (node.left != null && !visited.contains(node.left)) {
                visited.add(node.left);
                queue.offer(node.left);
            }

            // Explore right child
            if (node.right != null && !visited.contains(node.right)) {
                visited.add(node.right);
                queue.offer(node.right);
            }

            // Explore parent from map
            if (parentMap.containsKey(node) && !
visited.contains(parentMap.get(node))) {
                visited.add(parentMap.get(node));
                queue.offer(parentMap.get(node));
            }
        }
    }

    // All nodes left in queue are exactly K distance from target
    List<Integer> result = new ArrayList<>();
    while (!queue.isEmpty()) {
        result.add(queue.poll().val);
    }

    return result;
}
}

public class Main {
    public static void main(String[] args) {
        // Construct the binary tree
        TreeNode root = new TreeNode(3);
        root.left = new TreeNode(5);
        root.right = new TreeNode(1);
        root.left.left = new TreeNode(6);
        root.left.right = new TreeNode(2);
        root.left.right.left = new TreeNode(7);
        root.left.right.right = new TreeNode(4);
        root.right.left = new TreeNode(0);
        root.right.right = new TreeNode(8);

        // Node with value 5
        TreeNode target = root.left;
        int k = 2;

        Solution sol = new Solution();
        List<Integer> result = sol.distanceK(root, target, k);

        // Print the result
        for (int val : result) {
            System.out.print(val + " ");
        }
    }
}

```

Complexity Analysis

Time Complexity: $O(N)$, We visit each node exactly once when building the parent map using BFS, $O(N)$. We again visit each node at most once during the second BFS traversal from the target, $O(N)$. Hence, the total time complexity is $O(N)$, where N is the number of nodes in the binary tree.

Space Complexity: $O(N)$, The parent map stores one entry per node, $O(N)$. The queue and visited set used in BFS also take up to $O(N)$ space in the worst case. Therefore, the total space complexity is $O(N)$.